

SCADE Model Parser

Architecture Document

Version 1.0 | May 2026

Document Owner: Engineering Team

Classification: Internal

Status: Released

1 System Overview

The SCADE Model Parser is a single-module Python command-line tool. Its purpose is to bridge the gap between SCADE Suite model artefacts (.scade XML files) and general-purpose engineering tooling (Excel). It has no runtime dependencies beyond two well-maintained Python packages (openpyxl and defusedxml) and runs on any platform where Python 3.10+ is installed.

The tool is composed of four logical layers arranged in a pipeline:

CLI Layer Parses command-line arguments, validates paths, and orchestrates the pipeline.

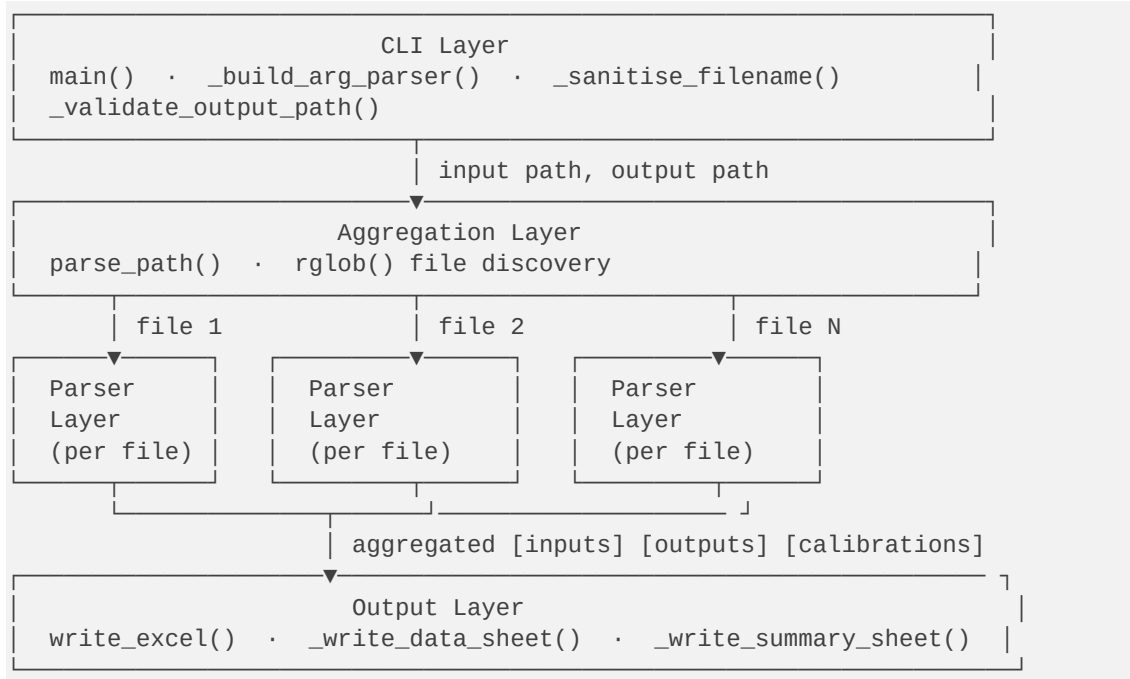
Aggregation Layer Discovers .scade files, invokes the parser for each, and merges results.

Parser Layer Performs XXE-safe XML parsing and depth-first element tree traversal to extract records.

Output Layer Formats and writes extracted records to a multi-sheet .xlsx workbook.

2 Component Architecture

The diagram below represents the component breakdown as a layered stack. Each layer communicates only with the layer immediately below it, ensuring clean separation of concerns.



2.1 CLI Layer

Entry point: `main()`. Responsibilities: argument parsing (`argparse`), absolute path resolution, output filename sanitisation, output directory writability check, and orchestration of the downstream layers. Returns an integer exit code for shell script integration.

2.2 Aggregation Layer

Function: `parse_path()`. Accepts a `Path` object that may be either a file or a directory. For directories, `rglob()` recursively discovers all `.scade` and `.xscade` files. For each discovered file, a fresh `ScadeModelParser` instance is created. Results are concatenated into three master lists which are returned to the caller.

Per-file errors (malformed XML, oversized files, permission denied) are caught as Python exceptions, reported to `stderr` with a `[SKIP]` tag, and processing continues with the remaining files.

2.3 Parser Layer

Class: `ScadeModelParser`. One instance per file. The `.parse()` method calls the XML loading function, then passes the document root to `_walk()` — a depth-first recursive traversal.

`_walk()` maintains two context variables — `operator` (the name of the current SCADE operator being visited) and `package` (the slash-delimited path of nested Package elements). These are threaded as parameters through each recursive call rather than stored as instance state, making the traversal reentrant and thread-safe per instance.

2.4 Output Layer

Functions: `write_excel()`, `_write_data_sheet()`, `_write_summary_sheet()`. Creates an `openpyxl` Workbook, adds a Summary sheet at index 0, then three data sheets. Each data sheet receives formatted headers, alternating row fills, auto-sized columns (capped at 55 characters), a frozen first row, and an auto-filter range. The workbook is saved atomically by `openpyxl`.

3 XML Parsing Strategy

3.1 Namespace Handling

SCADE XML files may or may not carry a namespace URI on element tags (e.g. {http://www.esterel-technologies.com/ns/scade/2}Operator). The helper function `_strip_ns()` removes any namespace prefix of the form {uri}TagName, leaving only the local tag name. This makes all element matching version-agnostic: the same tag strings (Input, Output, Sensor, Const, etc.) work regardless of whether a namespace is declared.

3.2 Attribute Resolution

SCADE attribute names for the same concept vary across versions and tool configurations (e.g. TypeRef vs typeRef vs Type for the data type, Direction vs direction for signal direction). The `_attr()` helper accepts a variadic list of candidate attribute names and returns the value of the first one present on the element. This strategy avoids brittle hard-coding of a single attribute name per field.

3.3 Multi-Pattern Signal Detection

SCADE operators declare their signal interfaces in at least three XML patterns. The parser handles all three in a single `_walk()` dispatch:

Pattern	XML Structure	Detection Logic
A — Flat Attribute	<code><Input Name="x" TypeRef="float32"/></code>	Detected when the Input or Output element itself has a Name attribute.
B — Variable Children	<code><Input><Variable Name="x" .../></Input></code>	Detected when Input/Output element contains Variable child elements.
C — Direction Attribute	<code><Variable ... Direction="in"/></code>	Detected when a Variable element has a Direction attribute of 'in' or 'out'.

When an Input or Output element with Variable children is encountered, the `_walk()` method processes the children inline and returns early (without re-entering them in the normal recursion loop) to avoid double-counting.

3.4 Calibration Parameter Detection

Three element tag names are matched as calibration parameters: Sensor, Const, and Constant. Both 'Const' and 'Constant' are matched explicitly because different SCADE Suite versions and project conventions use either spelling for compile-time constants. Matching only one would silently drop constants from some models. Default values are resolved by checking (in order): the Default attribute, the Value attribute, the InitVal attribute, then child elements named Value, Default, or InitVal. This multi-step resolution covers the range of formats observed in real SCADE models.

4 Security Architecture

Security is a first-class concern. The parser is designed to process untrusted model files safely in automated CI/CD pipelines.

Control	Risk Addressed	Severity	Implementation
XXE Injection Prevention	XML External Entity (XXE) attacks allow a malicious XML file to read arbitrary files from the host filesystem or trigger outbound network requests. The parser uses defusedxml as its primary defence: defusedxml raises an EntitiesForbidden exception on any DOCTYPE or entity declaration. When defusedxml is unavailable, a stdlib fallback disables the expat parser's ExternalEntityParserC reate handler and rejects DOCTYPE declarations.	High	defusedxml (primary), stdlib XXE rejection (fallback)
File Size Limit	Adversarially crafted XML files can consume disproportionate memory during parsing (billion-laugh / XML bomb attacks). The parser rejects any file exceeding MAX_FILE_SIZE_MB (default: 50 MB) before opening it. This is checked via Path.stat().st_size before any XML parsing begins.	High	Pre-parse size check in _validate_input_path()
Extension Allowlist	The parser only opens files whose extension is in ALLOWED_EXTENSIONS (.scade, .xscade). This prevents accidental processing of binary files or other XML dialects that could trigger	Medium	ALLOWED_EXTENSIONS set, checked in _validate_input_path()

	unexpected parser behaviour.		
Output Path Sanitisation	The output filename is passed through <code>_sanitise_filename()</code> , which removes any characters outside [A-Za-z0-9_.-]. The output path's parent directory must exist and be writable before any file is created. The output extension must be <code>.xlsx</code> .	Medium	<code>_sanitise_filename()</code> , <code>_validate_output_path()</code>
No Dynamic Code Execution	The codebase contains no calls to <code>eval()</code> , <code>exec()</code> , <code>compile()</code> , or <code>subprocess</code> with <code>shell=True</code> . All external interactions are file I/O through standard library functions and the <code>openpyxl</code> write path.	High	Code review — no dynamic execution
Path Traversal Prevention	All paths are resolved to absolute form using <code>Path.resolve()</code> before use. The output directory existence check prevents writing outside the intended directory even if a traversal sequence (<code>../</code>) appears in user input.	Medium	<code>Path.resolve()</code> on all paths

SECURITY WARNING — `defusedxml` fallback is a partial mitigation only: When `defusedxml` is not installed the parser falls back to the Python stdlib `xml.etree` module with a custom handler that rejects DOCTYPE declarations and external entity resolver creation. This blocks the most common XXE vector, but it does NOT provide the same depth of protection as `defusedxml`, which was purpose-built and audited for this threat class. In particular, some parser configurations or future Python versions may not honour the fallback hooks. Treat the stdlib fallback as a last resort for isolated development environments only. In any CI/CD pipeline, security scan, or production workflow, `defusedxml` MUST be installed. Run: `pip install defusedxml>=0.7.1`

5 Extension Points

The parser is designed so that common extensions require changes to only one location in the code.

5.1 Adding Support for a New SCADE Element Type

To extract a new element type (e.g. LocalVariable or GlobalVariable), add an elif branch in the `_walk()` method:

```
elif tag == 'LocalVariable':
    self._collect_var(element, 'Local', operator, package)
```

If the new element type has a different schema from Signal or Calibration records, add a new `_collect_xxx()` method and a corresponding list (`self.locals`, for example) in `__init__()`.

5.2 Adding a New Output Format

The aggregated inputs, outputs, and calibrations lists are plain Python lists of dicts. To add a CSV export:

```
import csv

def write_csv(inputs, outputs, calibrations, base_path: Path):
    for name, rows, cols in [
        ('inputs', inputs, _INPUT_COLS),
        ('outputs', outputs, _OUTPUT_COLS),
        ('calibrations', calibrations, _CALIBRATION_COLS),
    ]:
        with open(base_path.with_suffix(''). / f'{name}.csv', 'w') as f:
            writer = csv.DictWriter(f, fieldnames=cols)
            writer.writeheader()
            writer.writerows(rows)
```

5.3 Adjusting the File Size Limit

Change the `MAX_FILE_SIZE_MB` constant at the top of the module. No other code changes are required.

5.4 Adding New Attribute Name Variants

If a new SCADE version introduces a different attribute name for an existing field, add the new name to the appropriate `_attr()` call in `_collect_var()` or `_collect_cal()`. For example, if the type attribute is renamed "DataTypeRef" in a future version:

```
# Before
_attr(el, "TypeRef", "typeRef", "Type", "type", "DataType", default="unknown")

# After
_attr(el, "TypeRef", "typeRef", "Type", "type", "DataType", "DataTypeRef",
      default="unknown")
```


6 Technology Stack

Component	Technology	Version	Rationale
Runtime	Python	3.10+	Widely available in embedded engineering workflows; rich stdlib for XML and file I/O.
XML Parsing	defusedxml	>=0.7.1	Industry-standard XXE protection library. MIT licensed.
XML Fallback	xml.etree (stdlib)	Built-in	Used when defusedxml is unavailable; configured to reject DTD/entities.
Excel Generation	openpyxl	>=3.1.0	De-facto standard .xlsx library. Supports formatting, freeze panes, auto-filter.
CLI Parsing	argparse (stdlib)	Built-in	No additional dependency; sufficient for a simple two-argument interface.
Path Handling	pathlib (stdlib)	Built-in	Cross-platform path manipulation; used for all file system operations.

6.1 Dependency Policy

The parser deliberately minimises its dependency surface. Only two non-stdlib packages are required (openpyxl, defusedxml). This simplifies deployment in locked-down CI environments and reduces the attack surface from transitive dependencies. Future changes should maintain this principle: prefer stdlib over third-party packages unless there is a compelling reason.

7 Deployment and Integration

7.1 Standalone Use

The parser can be run directly with no build or installation step, provided Python 3.10+ and the two dependencies are available:

```
pip install openpyxl defusedxml
python scade_parser.py ./models/ --output signals.xlsx
```

7.2 CI/CD Pipeline Integration

In a CI pipeline (GitHub Actions, GitLab CI, Jenkins, etc.), the parser can be invoked as a post-build step to generate a signal inventory artefact:

```
# Example GitHub Actions step
- name: Extract SCADE signals
  run: |
    pip install openpyxl defusedxml
    python scade_parser.py ${GITHUB_WORKSPACE}/models/ \
      --output signals_${GITHUB_SHA}.xlsx
- uses: actions/upload-artifact@v4
  with:
    name: scade-signals
    path: signals_*.xlsx
```

7.3 Programmatic (Library) Use

`parse_path()` and `ScadeModelParser` can be imported and used as a library without going through the CLI:

```
from scade_parser import parse_path, write_excel
from pathlib import Path

inputs, outputs, calibrations = parse_path(Path('./models'))
# Post-process records as needed ...
write_excel(inputs, outputs, calibrations, Path('out.xlsx'))
```

7.4 Supported Platforms

Platform	Status	Notes
Windows 10/11	Supported	Default line endings handled by Python; .scade files on network drives supported.
macOS 12+	Supported	No known platform-specific issues.
Linux (Ubuntu 20+)	Supported	Primary development and CI target.
Python 3.10	Minimum	Uses type hint syntax (list[dict]) requiring 3.10+.
Python 3.12	Tested	All features verified on CPython 3.12.

8 Revision History

Version	Date	Author	Summary
1.0	May 2026	Engineering Team	Initial release.